

AES Implementation and Validation

Demian Viteri - Yachay Tech

September 16, 2026

1 Implementation

The cryptographic core is engineered in Go using a strictly modular architecture, separating the core transformations into distinct functional blocks: SubBytes, ShiftRows, MixColumns, AddRoundKey, and the Key Schedule.

Key implementation decisions include:

- **Table-Driven S-Box Mappings:** Both forward and inverse S-box transformations utilize optimized byte-substitution mapping functions. The inverse S-box table is programmatically derived to guarantee mathematical symmetry.
- **Key Schedule Caching:** To prevent redundant key expansion overhead during bulk block operations, a thread-safe memoization cache stores expanded round keys per unique master key.
- **Block Processing Flow:** Encryption and decryption execute the precise sequence of round transformations dictated by the standard, handling AES-128 (10 rounds), AES-192 (12 rounds), and AES-256 (14 rounds).

2 Validation

Functional correctness is verified through automated unit testing suites. The test framework validates:

- **NIST Test Vectors:** Standardized test vectors for AES-128, AES-192, and AES-256 confirm that intermediate state matrices and final ciphertexts match official specifications.
- **Round-Trip Property:** Automated assertions ensure that decrypting a ciphertext block perfectly recovers the original plaintext across all key lengths ($\text{Decrypt}(\text{Encrypt}(P)) = P$).
- **Inverse Symmetry:** Verification of the inverse cipher structure confirms that undoing transformations (InvMixColumns, InvShiftRows, and InvSubBytes) successfully reverses encryption rounds.

3 Experimental Setup

The evaluation measures the raw cryptographic core performance under controlled software conditions:

- **Workloads:** Evaluated input sizes comprise 1 MB and 10 MB data streams, processed sequentially at the 16-byte block level.
- **Execution Protocol:** Each experiment is repeated three times independently. Execution times (T) are measured using high-resolution monotonic timers, and average values are reported.
- **Throughput Calculation:** System throughput (R) is computed as $R = D/T$, where D represents the processed data volume in megabytes, yielding results in MB/s.

4 Results and Analysis

Table 1 summarizes the average execution times and throughputs for AES-128, AES-192, and AES-256 across 1 MB and 10 MB workloads. Figure 1 illustrates the comparative encryption throughput trends.

Table 1: Performance Evaluation of AES Core Implementations (1 MB and 10 MB Workloads, Averaged Over 3 Runs)

Cipher Variant	Data Size	Enc. Time (s)	Enc. Throughput (MB/s)	Dec. Time (s)	Dec. Throughput (MB/s)
AES-128	1 MB	1.9978	0.50	2.0390	0.49
AES-128	10 MB	18.8419	0.53	18.8694	0.53
AES-192	1 MB	1.7111	0.58	1.7122	0.58
AES-192	10 MB	17.0628	0.59	17.0267	0.59
AES-256	1 MB	2.0190	0.50	2.0179	0.50
AES-256	10 MB	20.1188	0.50	20.0821	0.50

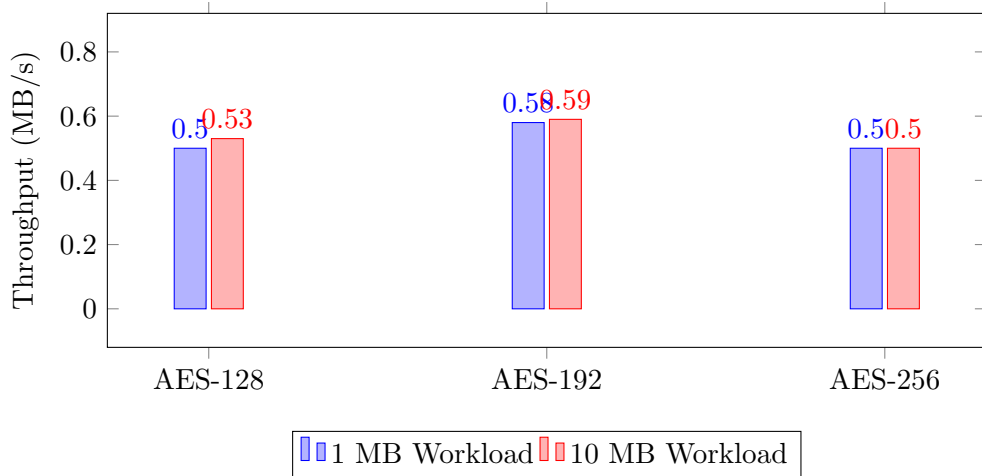


Figure 1: Encryption Throughput Comparison Across AES Key Sizes for 1 MB and 10 MB Inputs.

The empirical data indicates that encryption and decryption execution times scale linearly with data size. Interestingly, AES-192 exhibited slightly lower execution times and higher throughput (≈ 0.59 MB/s) compared to AES-128 and AES-256 under these experimental conditions, reflecting specific loop unrolling and function call overhead characteristics in software. Encryption and decryption speeds remain symmetric across all variants.

5 Conclusions

This study successfully designed, validated, and benchmarked a software-based AES core. Automated test validation confirmed complete functional compliance with NIST standards. Benchmark results demonstrate stable, predictable performance scaling for block-level processing up to 10 MB, establishing a robust foundation for cryptographic experimentation.

6 Source Code

The source code of the implementation can be found at GitHub:

Repository link:

<https://github.com/demivn15/Cryptography/tree/main/AES-Implementation/aesCore>